- Describes the context—such as a particular class, method, or situation—of what you're testing
- Provides a Ruby class to act as a scope for your shared logic, such as hooks, let definitions, and helper methods
- Runs common setup and teardown code shared by several examples

Let's look at a few different ways to create these groups.

The Basics

We saw the basic ways to organize specs in *Groups, Examples, and Expectations*, on page 5:

- describe creates an example group.
- it creates a single example.

Before we move on to other ways to organize specs, let's discuss the finer points of these building blocks.

describe

All of the different variations on describe amount to the same thing: you say what it is you're testing. The description can be a string:

07-structuring-code-examples/01/getting_the_words_right.rb
```
RSpec.describe 'My awesome gardening API' do
end
```

…or any Ruby class, module, or object:

07-structuring-code-examples/01/getting_the_words_right.rb
```
RSpec.describe Perennials::Rhubarb do
end

RSpec.describe Perennials do
end

RSpec.describe my_favorite_broccoli do
end
```

You can combine these two approaches, and pass in a Ruby class/module/object followed by a string:

07-structuring-code-examples/01/getting_the_words_right.rb
```
RSpec.describe Garden, 'in winter' do
end
```

The differences may seem subtle, but passing a class name has some advantages. It requires the class to exist and to be spelled correctly—which will help you catch mistakes. It also allows RSpec extension authors to provide